



CHALLENGE 18 / 20

THE INSIDER WHO KNEW THE FORENSICS

DETECTING DELIBERATE EVIDENCE AVOIDANCE

NATIONAL LEVEL HACKATHON • 2026

POWERED BY SYNTHOQUEST PVT LTD

Student Project Development Challenge • No organizer-provided evidence package.

1 | PROBLEM BACKGROUND

A fintech with 14 million customers found 2.3 million of its customer records on a broker's list. The data was fresh enough to have left within the last 60 days. A senior database administrator of nine years, who helped design the company's audit and loss-prevention controls, had resigned three weeks before. Query audit, data-loss tools, privileged-access recordings and backup access all show nothing unusual.

But some audit gaps line up with maintenance windows he scheduled. A log-pipeline migration he led dropped a field. A monitoring threshold he tuned sits just above normal activity. Three privileged-session recordings are missing, attributed to storage failure.

Counsel must demonstrate deliberate evasion without arguing that he is guilty because he knew how. The people who designed the controls are all suspects.

2 | PROBLEM STATEMENT

Develop an **Evidence Avoidance Detection System** that models what each class of activity should leave across every layer of the environment, finds missing, altered or attenuated evidence, and assigns each gap an explanation distribution across fault, misconfiguration, normal variation and deliberate avoidance.

The system must identify whether the gaps form a coherent concealment strategy, and link them to candidate actors through capability and access rather than motive, with explicit controls against false accusation.

Hard constraints

- There is no positive evidence of exfiltration, and some gaps are genuine faults.
- The adversary designed the controls and knows what they see.
- Circular reasoning, such as knowing forensics therefore guilty, must be avoided and shown to be avoided.
- False-accusation risk must be quantified for every actor named.
- The system must work for activity types it was not built around.

No evidence package or dataset is provided. Teams decide what to build, what data they need, how to simulate the environment and how to validate the result. No tools or architecture are prescribed.

3 | MANDATORY FEATURES

Teams must develop a working software solution. The system must provide the following features. How each is achieved is for the team to decide.

01	Multi-Layer Footprint Modelling	Ability to model what activity should leave across layers.
02	Expected-Evidence Gap Detection	Ability to find evidence that should exist and does not.
03	Gap Cause Attribution	Ability to weigh fault, misconfiguration, variation and avoidance.
04	Concealment Strategy Inference	Ability to find a coherent strategy across separate gaps.
05	Control-Knowledge Exploitation Analysis	Ability to detect behaviour that exploits knowledge of the controls.
06	Capability-Based Actor Linking	Ability to link gaps to actors by access and capability.
07	False-Accusation Control	Ability to quantify and limit the risk of naming the wrong person.
08	Novel-Activity Generalisation	Ability to handle activity types not seen before.

The system must clearly distinguish between:

PRESENT	EXPECTED & MISSING	LOGGING FAILURE	DELIBERATE AVOIDANCE	NORMAL VARIATION
---------	--------------------	-----------------	----------------------	------------------

A gap that cannot be explained must not be dismissed as normal.

Build, don't search. This is a development challenge. Teams must create the system and the controlled environment that proves it works. Submitting an analysis of a given dataset is not a solution.

4 | EXPECTED OUTPUT

Live demonstration. The team must show that its system can handle a complex controlled scenario created by the team. The demonstration must show:

01	Teams' own simulated company with layered logging and several insiders
02	A planted low-footprint exfiltration among genuine faults
03	Detection of missing evidence
04	Cause attribution for each gap
05	Recovery of a concealment strategy
06	Capability-based candidate list with false-accusation risk
07	Generalisation to an unseen activity type

The team must be able to explain why the system reached each major conclusion.

Validation standard (every team)

- Build a scenario generator with known ground truth and evaluate on at least ten distinct scenarios, including ones with deliberately misleading evidence.
- At the demonstration, judges change scenario parameters on the spot. The system must adapt without being rebuilt.
- Include an adversary variant that actively tries to defeat the system, and report how the system held up.
- Stated confidence must match measured accuracy. Overconfident or unexplained conclusions lose marks.
- No real systems, real personal data or functional malware may be used.

Submission. Each team must submit:

01	Working Prototype	A functional implementation of the proposed solution.
02	Source Code	Complete source code required to reproduce the project.
03	Technical Architecture	Architecture and component description.
04	Validation Environment	A controlled environment created by the team to demonstrate the solution.
05	Technical Documentation	Implementation, design and testing documentation.
06	Forensic Investigation Report	A professional report generated from the demonstrated investigation.
07	Demonstration	Live technical demonstration to the judging panel.
08	Limitations	Known limitations and situations where the system cannot establish a reliable conclusion.